

Contrato Frontend ↔ Backend — Participante

Base da API

Em ambiente local:

`http://localhost:3000`

Depois, quando o backend estiver publicado, só muda a URL base. As rotas permanecem iguais. O servidor Express monta as rotas de participante em `/participant`.

Autenticação

Todas as rotas de participante usam:

`Authorization: Bearer <accessToken>`

O `accessToken` identifica o participante e é único. Ele será fornecido quando o participante for criado na sessão.

1. Consultar estado atual

`GET /participant/me`

Essa deve ser a principal rota do frontend. Ela informa em qual tela o participante deve estar.

Exemplo:

```
{
  "participant": {
    "id": "sp-001",
    "slot": "P1",
    "displayName": "Alice",
    "participantCode": "G1P1",
    "joinedAt": "2026-09-03T...",
    "lastSeenAt": "2026-09-03T...",
    "createdAt": "2026-09-03T..."
  },
  "session": {
    "id": "session-id",
    "name": "Turma A",
    "status": "IN_PROGRESS"
  },
}
```

```
"stage": "JUDGMENT",
"currentAttempt": {
  "id": "attempt-id",
  "endowment": 16,
  "distributorDistribution": 12,
  "receptorDistribution": 4,
  "distributorCharacter": "Lucas",
  "receptorCharacter": "Isaac"
},
"trialResult": null
}
```

O frontend **não deve tentar calcular o estágio da rodada**. Deve renderizar a interface a partir de **stage**. O backend já faz essa derivação.

2. Valores possíveis de **stage**

WAITING_SESSION

Sessão ainda não iniciada. Mostrar tela de espera.

JUDGMENT

Mostrar os cards e a pergunta de **Justo ou Injusto**.

WAITING_JUDGMENT_PARTNER

O participante já respondeu e deve aguardar o parceiro.

PUNISHMENT

Ambos já responderam Justo/Injusto. Mostrar a pergunta de **Punir ou Não Punir**.

WAITING_PUNISHMENT_PARTNER

O participante já decidiu sobre punição e aguarda o parceiro/finalização.

RESULT

A rodada terminou e **trialResult** estará preenchido. Mostrar as consequências da rodada.

WAITING_RESULT_PARTNER

O participante já terminou de ver/confirmou seu resultado e aguarda o parceiro.

COMPLETED

Experimento encerrado. Mostrar tela final.

Essa sequência está codificada no backend como:

JUDGMENT → WAITING_JUDGMENT_PARTNER → PUNISHMENT →
WAITING_PUNISHMENT_PARTNER → RESULT → WAITING_RESULT_PARTNER →
próxima rodada/COMPLETED.

3. Enviar julgamento Justo/Injusto

POST /participant/attempt/:attemptId/judgment

Body:

```
{  
  "judgment": "Just"  
}
```

ou:

```
{  
  "judgment": "Unjust"  
}
```

Usar sempre o `currentAttempt.id` recebido pelo GET /participant/me.

A resposta já é novamente o **estado atualizado completo do participante**. Portanto, depois do POST, o frontend pode simplesmente substituir seu estado pelo JSON retornado.

4. Enviar decisão de punição

POST /participant/attempt/:attemptId/punishment

Body:

```
{  
  "punishment": "Punish"  
}
```

ou:

```
{  
  "punishment": "NoPunish"  
}
```

Novamente, a resposta é o estado atualizado completo.

O backend só aceita a decisão de punição depois que **os dois participantes já responderam Justo/Injusto**. Eles não precisam concordar no julgamento.

Feedback imediato ao escolher Punish

No frontend, quando o POST de **Punish** for aceito com sucesso, mostrar imediatamente o feedback individual de que o participante gastou 1 moeda e tocar o som negativo correspondente.

Não esperar o parceiro para esse feedback visual/sonoro individual.

A consequência final da punição sobre o personagem, porém, só aparece no **RESULT**.

5. Resultado da rodada

Quando:

`stage === "RESULT"`

`trialResult` estará preenchido:

```
{
  "ownIndividualCost": 1,
  "ownCoinsAfter": 79,
  "punishmentApplied": true,
  "distributorResult": {
    "character": "Lucas",
    "finalCoins": 4,
    "coinsLost": 8
  },
  "culturalConsequence": 3,
  "groupCoinsAfter": 3
}
```

Os campos significam:

- `ownIndividualCost`: quanto o próprio participante perdeu naquela rodada;
- `ownCoinsAfter`: saldo individual depois da rodada;
- `punishmentApplied`: `true` somente quando **os dois escolheram Punish**;
- `distributorResult`: consequência sobre o distribuidor; só existe quando `punishmentApplied=true`;
- `culturalConsequence`: moedas de grupo recebidas naquela rodada; atualmente 0 ou 3;
- `groupCoinsAfter`: saldo acumulado da dupla.

`punishmentApplied` e `culturalConsequence` são **coisas diferentes**. Pode existir consequência cultural mesmo sem punição aplicada. O formato público desse resultado é deliberadamente limitado a esses campos.

Se:

"punishmentApplied": false

então:

"distributorResult": null

Nesse caso, **não mostrar Card 4 de punição**.

6. Confirmar que terminou de ver o resultado

Depois de apresentar todas as consequências da rodada:

`POST /participant/attempts/:attemptId/result/acknowledge`

Não precisa body.

Exemplo:

```
fetch(`${API_URL}/participant/attempts/${attemptId}/result/acknowledge`, {
  method: "POST",
  headers: {
    Authorization: `Bearer ${token}`
  }
})
```

A resposta também é o estado atualizado. Ela normalmente será:

`WAITING_RESULT_PARTNER`

ou, se o parceiro já confirmou:

`JUDGMENT`

da próxima rodada.

Na última rodada, após os dois acknowledgements, a sessão chega a `COMPLETED`.

7. Sincronização dos dois participantes

Por enquanto **não depende de Socket.IO**.

Nas telas de espera:

- `WAITING_SESSION`
- `WAITING_JUDGMENT_PARTNER`
- `WAITING_PUNISHMENT_PARTNER`
- `WAITING_RESULT_PARTNER`

fazer polling de:

`GET /participant/me`

Sugestão inicial:

a cada 1000–1500 ms

Quando `stage` mudar, parar de mostrar a espera e renderizar a nova etapa.

O `GET /participant/me` também serve para **recovery**: se a criança atualizar a página no meio do experimento, consultar essa rota e reconstruir a tela a partir do estado devolvido.

8. Regra importante de navegação

Não manter o fluxo experimental apenas no estado local do React.

`stage` recebido do backend é a fonte de verdade.

Por exemplo, não fazer:

`currentStep++`

para decidir qual tela vem depois.

Fazer:

```
switch (state.stage) {  
  case "JUDGMENT":  
  case "WAITING_JUDGMENT_PARTNER":  
  case "PUNISHMENT":  
  case "WAITING_PUNISHMENT_PARTNER":  
  case "RESULT":  
  case "WAITING_RESULT_PARTNER":  
  case "COMPLETED":  
}
```

Assim refresh, reconexão e diferença de velocidade entre P1/P2 continuam funcionando.

9. Dados que o frontend NÃO receberá

De propósito, o participante não recebe:

- condição A/B/C;
- variante ABAC/ACAB/BCBC/CBCB;
- bloco atual;
- número da rodada;
- total de rodadas;
- rodadas restantes;
- `culturant`;
- julgamento do parceiro;
- decisão de punição do parceiro;
- `accessToken` de qualquer participante.

Portanto, **não criar na interface elementos como “Rodada 12 de 64” ou “faltam 20 rodadas”**. Esses dados não fazem parte da experiência apresentada às crianças e o backend deliberadamente não os expõe.

10. Erros HTTP

A API retorna JSON no formato:

```
{
  "error": "mensagem"
}
```

Códigos principais:

`400` — body/valor inválido

`401` — token ausente, malformado ou inválido

`404` — recurso/attempt não encontrado

`409` — ação válida em formato, mas inválida no estado atual do experimento

`500` — erro inesperado do servidor.

Em `409`, o mais seguro no frontend é consultar novamente:

`GET /participant/me`

e sincronizar a tela com o estado verdadeiro do backend.

Resumo do fluxo

GET /participant/me
↓
WAITING_SESSION
↓
JUDGMENT
↓ POST judgment
WAITING_JUDGMENT_PARTNER
↓ polling
PUNISHMENT
↓ POST punishment
WAITING_PUNISHMENT_PARTNER
↓ polling
RESULT
↓ POST result/acknowledge
WAITING_RESULT_PARTNER
↓ polling
JUDGMENT da próxima tentativa
...
COMPLETED

Por enquanto, para desenvolver o frontend, ela pode **mockar exatamente esses JSONs e stages**. Depois, quando conectarmos os dois projetos, basta trocar os mocks pelas chamadas HTTP reais.